Universidad de Chile
Facultad de Ciencias Físicas y Matemáticas
Departamento de Ingeniería Eléctrica

Tarea N°3

Selección de Características

Integrantes: Diego Vega
Profesor: Javier Ruiz del Solar
Auxiliar: Patricio Loncomilla
Ayudantes: Benjamín Llancao Mella
Bruno Cavieres L.
Gonzalo Olguín M.

Fecha de realización: 24 de mayo de 2024
Fecha de entrega: 21 de mayo de 2024
Santiago de Chile

Introducción

En el presente informe se detalla el desarrollo de la tarea Nº3 del curso Inteligencia Computacional. El desarrollo de la tarea se hizo acorde a las indicaciones del documento *Tarea 3 EL4106 2024.pdf*.
En el documento mencionado anteriormente se tiene por objetivo el estudio de la selección de características de el conjunto de datos **Heart Disease**, el cual fue entregado por el cuerpo docente en el archivo *processed.cleveland.data*. El Data Set proviene del **UC Irvine Machine Learning Repository** y posee 13 columnas, las primeras 11 corresponden a las características mientras que la última corresponde al elemento que se desea predecir. Durante el estudio de la selección de características se utilizaran dos clasificadores: **SVM Lineal** y **Random Forest**.
Los objetivos de la tera, y por tyanto lo que se ha realizado, consisten en:

Selección de Características y Entrenamiento Esta sección consiste en: La lectura del Data Set y la modificación del mismo. División del Data Set en tres conjuntos representativos: el de entrenamiento (60 %), el de validación (20 %) y el de prueba (20 %). Normalización de los subconjuntos mediante *StandarScaler*. Luego, se obtendrá un **SVM Lineal** mediante el uso de *GridSearchCV*, posteriormente se entrenara dicho **SVM Lineal** con el conjunto de entrenamiento, finalmente se evaluara el clasificador mediante el valor del accuracy y la matriz de confusión ambos a partir del conjunto de validación. Posteriormente, se crearan nuevos clasificadores a partir del reentrenamiento mediante el uso de selectores de las características *SelectFromModel* y *SelectKBest*, en donde se utilizara $k = 6$ y $k = 6$ para el ultimo selector.
Lo anterior se detallara en la sub-sección **Selección de Características y Entrenamiento** del **Desarrollo** Finalmente se repetirá lo realizado para el clasificador **SVM Lineal** pero, para un clasificador **Random Forest** con max_depth = 8.

Análisis Esta sección consiste en: La evaluación, de manera general, de los clasificadores, indicando cual entrega mejores resultados.
Además, se indicara el efecto de reducir las características sobre la precisión y sobre el tiempo de entrenamiento, para lo cual se utilizaran gráficos.
Finalmente, analizara la utilizad de la reducción de características para el conjunto de datos utilizado.
Lo anterior se detallara en la sub-sección **Análisis** del **Desarrollo**

Índice de Contenidos

1. Desarrollo **1**
1.1. Selección de Características y Entrenamiento . . . 1
1.1.1. a) . . . 1
1.1.2. b) . . . 2
1.1.3. c) . . . 2
1.1.4. d) . . . 3
1.1.5. e) . . . 4
f) . . . 5
1.1.7. g) . . . 5
1.1.8. h) . . . 7
1.1.9. i) . . . 9
1.1.10. j) . . . 11
1.1.11. k) . . . 12
1.1.12. l) . . . 14
1.1.13. m) . . . 16
1.1.14. n) . . . 18
1.2. Análisis . . . 19
1.2.1. a) . . . 19
1.2.2. b) . . . 19
1.2.3. c) . . . 21

2. Conclusiones **21**
Aprendizajes . . . 21
Dificultades . . . 21
Evaluación de resultados . . . 22

Anexo Código **22**

Índice de Figuras

1. Matriz de confusión normalizada del mejor clasificador SVM lineal. . . . 4
2. Matriz de confusión normalizada del mejor clasificador SVM lineal según *SelectFromModel*. . . . 6
3. Matriz de confusión normalizada del mejor clasificador SVM lineal según las características de *SelectKBest* con $k = 6$. . . . 8
4. Matriz de confusión normalizada del mejor clasificador SVM lineal según las características de *SelectKBest* con $k = 3$. . . . 10
5. Matriz de confusión normalizada del mejor clasificador RandomForest. . . . 12
6. Matriz de confusión normalizada del mejor clasificador RandomForest con *SelectFromModel*. . . . 14

7. Matriz de confusión normalizada del mejor clasificador RandomForest con *SelectKBest* con $k = 6$. 16
8. Matriz de confusión normalizada del mejor clasificador RandomForest con *SelectKBest* con $k = 3$. 18
9. Comparación de las matrices de confusión de los mejores clasificadores. 19
10. Gráficas para el clasificadores SVM. 20
11. Gráficas para el clasificadores RandomForest. 20

Índice de Tablas

1. Mejor clasificador SVM lineal y su tiempo de entrenamiento. 4
2. Accuracy SVM. 4
3. Mejor clasificador SVM lineal según *SelectFromModel* y su tiempo de entrenamiento. 6
4. Accuracy SVM con *SelectFromModel*. 7
5. Mejor clasificador SVM lineal según las características de *SelectKBest* (con $k = 6$) y su tiempo de entrenamiento. 7
6. Accuracy SVM con *SelectKBest* con $k = 6$. 8
7. Mejor clasificador SVM lineal según las características de *SelectKBest* con $k = 3$ y su tiempo de entrenamiento. 9
8. Accuracy con *SelectKBest* con $k = 3$. 10
9. Mejor clasificador RandomForest y su tiempo de entrenamiento. 11
10. Accuracy RandomForest 12
11. Mejor clasificador RandomForest con *SelectFromModel* y su tiempo de entrenamiento. 13
12. Accuracy RandomForest con *SelectFromModel*. 14
13. Mejor clasificador RandomForest con *SelectKBest* con $k = 6$ y su tiempo de entrenamiento. 15
14. Accuracy RandomForest con *SelectKBest* con $k = 6$ 16
15. Mejor clasificador RandomForest con *SelectKBest* con $k = 3$ y su tiempo de entrenamiento. 17
16. Accuracy RandomForest con *SelectKBest* con $k = 3$. 18
17. Comparación de Accuracy. 19

Índice de Códigos

1. Carga y modificación del Data Set. 1
2. División del *DataFrame*. 2
3. Normalización de los subconjuntos de entrenamiento, validación y prueba. . . 2
4. Obtención del mejor clasificador SVM con kernel lineal. 3
5. *SelectFromModel* clasificador SVM. 5

6. Obtención del mejor clasificador SVM lineal según las características obtenidas mediante *SelectFromModel*. 5
7. Obtención del mejor clasificador SVM lineal según las características obtenidas mediante *SelectKBest* con $k = 6$. 7
8. Obtención del mejor clasificador SVM lineal según las características obtenidas mediante *SelectKBest* con $k = 3$. 9
9. Obtención del mejor clasificador RandomForest con max_depth = 8. 11
10. Obtención del mejor clasificador RandomForest según las características obtenidas mediante *SelectFromModel*. 13
11. Obtención del mejor clasificador RandomForest con *SelectKBest* con $k = 6$. . 15
12. Obtención del mejor clasificador RandomForest con *SelectKBest* con $k = 3$. . 17
A.1. Código de la tarea. 22

1. Desarrollo

1.1. Selección de Características y Entrenamiento

1.1.1. a)

En la presente sub-sección se realiza la carga de Data Set, para lo cual se utiliza la librería *pandas*. Además, como el Data Set contenido en el archivo *processed.cleveland.data* no contiene los nombres de las columnas/características, estos deben ser agregados de manera manual. Las columnas agregadas fueron ['age', 'sex', 'cp', 'trestbps', 'chol', 'fbs', 'restecg', 'thalach', 'exang', 'oldpeak', 'slope', 'ca', 'thal', 'num'] , dichas características son explicadas en 1.1.1. Luego la columna *num* que posee un intervalo $[0, 4]$ se utiliza para crear una nueva columna llamada *class*, la cual poseerá un valor 1 si num ≥ 2 y valor 0 en caso contrario, eliminando en el proceso a la columna *num*. Lo anterior es debido a que el clasificador **SVM** utilizado solo permite la clasificación binaria.

age La edad.

sex El sexo.

cp Tipo de dolor de pecho.

trestbps Presión durante reposo.

chol Colesterol sérico.

fbs Azúcar en la sangre durante ayuno.

restecg Resultados del electrocardiograma en reposo.

thalach Máximos ritmo cardiaco alcanzado.

exang Angina inducida por el ejercicio.

oldpeak Depresión ST inducida por el ejercio relativo al reposo.

slope La pendiente del ejercicio pico en el segmento ST.

ca Cantidad de vasos tintados por fluoroscopia.

thal Valor numérico que indica si existe algún defecto.

El código utilizado para la carga y modificación del Data Set se presenta en 1.

Código 1: Carga y modificación del Data Set.

```
1 DataSet = 'processed.cleveland.data'
2 Columns = ['age', 'sex', 'cp', 'trestbps', 'chol', 'fbs', 'restecg', 'thalach', 'exang', 'oldpeak', '
      ↪ slope', 'ca', 'thal', 'num']
```

```
DataFrame = pd.read_csv(DataSet, names=Columns)
def high_or_low(num):
  if num >= 2:
    return 1
  else:
    return 0
DataFrame['class'] = DataFrame['num'].apply(high_or_low)
DataFrame = DataFrame.drop('num', axis=1)
DataFrame = DataFrame.replace('?', np.nan)
DataFrame = DataFrame.dropna()
```

Con lo cual el *DataFrame* utilizado queda con una columna de clase binaria.

1.1.2. b)

En la presente sub-sección se divide el *DataFrame* en tres conjuntos representativos, el de entrenamiento, el de validación y el de prueba, con porcentajes del 60 %, 20 % y 20 % respectivamente.
Para realizar la separación se utilizo dos veces la función *train_test_split* de *scikit-learn*, utilizando un *random_state* fijo y *stratify* para mantener la proporción de las clases.
El código utilizado para realizar la división del *DataFrame* se adjunta en 2.

Código 2: División del *DataFrame*.

```
X = DataFrame.drop('class', axis=1)
y = DataFrame['class']
TrainRatio = 0.6
TestRatio = 0.2
ValidationRatio = 0.2
XTrain, XTemporary, yTrain, yTemporary = train_test_split(X, y, test_size=1-TrainRatio,
    ↪ random_state=3, stratify=y)
XValidation, XTest, yValidation, yTest = train_test_split(XTemporary, yTemporary,
    ↪ test_size=0.5, random_state=3, stratify=yTemporary)
```

1.1.3. c)

En la presente sub-sección se normalizan las características de los 3 subconjuntos creados en 1.1.2, es decir, se normalizan los subconjuntos de entrenamiento, validación y prueba. Para lo anterior, se utilizo la función *StandarScaler* de *scikit-learn*. La función fue entrenada con el conjunto de entrenamiento y posteriormente se utilizo sobre los conjuntos de entrenamiento, validación y prueba, obteniendo así los conjuntos normalizados.
El código utilizado para realizar la normalización de los datos se adjunta en 3.

Código 3: Normalización de los subconjuntos de entrenamiento, validación y prueba.

```
Scaler = StandardScaler()
```

```
Scaler.fit(XTrain)
XTrainNormalized = Scaler.transform(XTrain)
XValidationNormalized = Scaler.transform(XValidation)
XTestNormalized = Scaler.transform(XTest)
```

Es importante mencionar que durante el resto del desarrollo se utilizaron los subconjuntos normalizados.

1.1.4. d)

En la presente sub-sección se creo y entreno un clasificador Support Vector Machine con kernel lineal. Además, para la elección del clasificador se utilizo *GridSearchCV* de *scikit-learn* para encontrar el mejor hiper-parámetro **C**.
Para no hacer uso de la validación cruzada se utilizo la función *PredefinedSplit* de *scikit-learn*, la cual permite definir una división/pliegues de los datos a utilizar durante la validación cruzada. En este caso particular, se utilizo para señalar que muestras pertenecían al subconjunto de entrenamiento y cuales al de validación.
Posterior al entrenamiento realizado por *GridSearchCV*, se vuelve a entrenar el mejor clasificador (el que se obtiene con el método *best_estimator* de *GridSearchCV*) con el subconjunto de entrenamiento.
Por ultimo, se calcula el tiempo de entrenamiento.
El código utilizado para la creación y entrenamiento del clasificador SVM con kernel lineal se adjunta en 4.

Código 4: Obtención del mejor clasificador SVM con kernel lineal.

```
Indexs = np.zeros(len(XTrainNormalized) + len(XValidationNormalized))
Indexs[:len(XTrainNormalized)] = -1
PreDefinedSplit = PredefinedSplit(test_fold=Indexs)
ParametersGrid1 = {'C': [0.01, 0.1, 1, 10, 100]}
SVM = SVC(kernel='linear')
GridSearch0 = GridSearchCV(estimator=SVM, param_grid=ParametersGrid1, cv=
    ↪ PreDefinedSplit)
InitialTrainingTime = time.time()
GridSearch0.fit(np.vstack((XTrainNormalized, XValidationNormalized)), np.hstack((yTrain,
    ↪ yValidation)))
FinalTrainingTime = time.time()
BestSVM0 = GridSearch0.best_estimator_
BestSVM0.fit(XTrainNormalized, yTrain)
TrainingTime0 = FinalTrainingTime - InitialTrainingTime
```

Los resultados obtenidos mediante la ejecución de 4 se presentan en 1.

Tabla 1: Mejor clasificador SVM lineal y su tiempo de entrenamiento.

Mejor clasificador SVM lineal	Tiempo de entrenamiento
SVC(C=0.01, kernel=linear)	0.44524049758911133

1.1.5. e)

En la presente sub-sección se evalúo el mejor clasificador obtenido en 1.1.4, para lo cual se hizo uso de dos métricas: la matriz de confusión y el accuracy. Ambas fueron calculadas mediante una predicción sobre el subconjunto de validación. Además, para generar la gráfica de la matriz de confusión se utilizo *heatmap* de *seaborn*.
Lo obtenido se presenta a continuación.

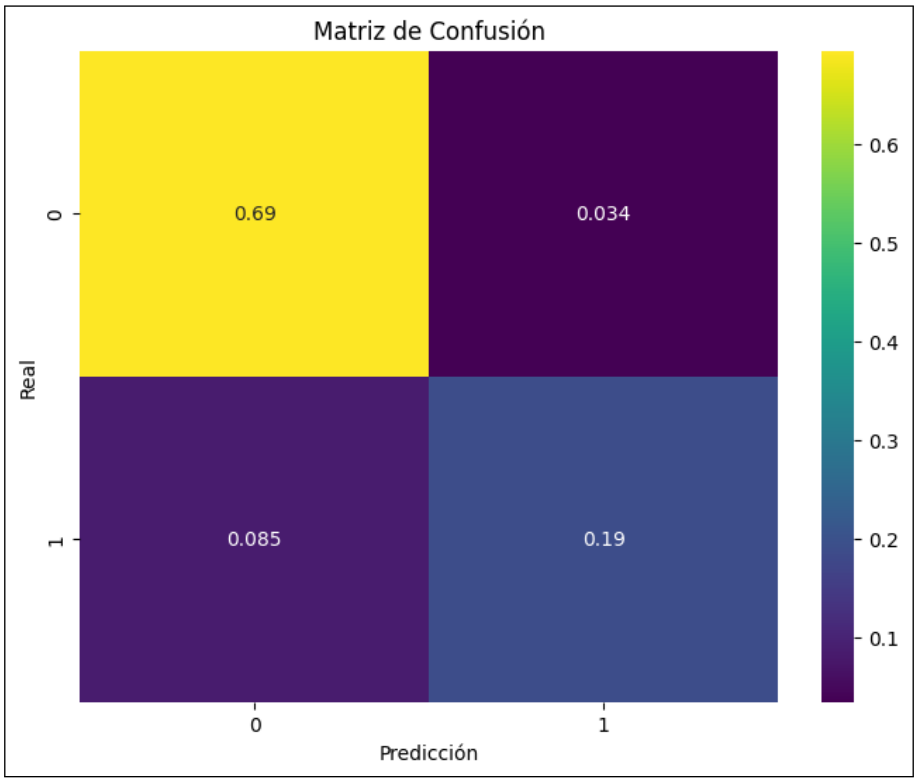


Figura 1: Matriz de confusión normalizada del mejor clasificador SVM lineal.

Tabla 2: Accuracy SVM.

Accuracy
0.8813559322033898

De la matriz de confusión normalizada, representada en 1, es posible observar que el valor de TN es superior al resto, incluso representando más de la mitad del total de casos, le sigue

el valor de TP, luego el valor de FN y por ultimo el de FP, por lo que el clasificador obtenido es «bueno», ya que los primeros dos valores mencionados son aciertos y suman 88 %, mientras que el 12 % restante son errores.
Del valor del accuracy, representado en 2, es posible observar que es bastante alto, lo cual implica un clasificador «bueno».

f)

En la presente sub-sección se utilizo *SelectFromModel* de *scikit-learn* sobre el mejor clasificador encontrado en 1.1.4, proceso por el cual se seleccionan las características mas relevantes (según el algoritmo de la función). El código utilizado para la creación del selector y la obtención de las mejores características se adjunta en 5.

Código 5: *SelectFromModel* clasificador SVM.

```
Selector1 = SelectFromModel(BestSVM0)
SelectedFeatures1 = X.columns[Selector1.get_support()]
```

Las mejores características según *SelectFromModel* son:

1. cp.
2. thalach.
3. oldpeak.
4. ca.
5. thal.

1.1.7. g)

En la presente sub-sección se entreno un clasificador SVM lineal, pero ahora con el conjunto de características reducido. El conjunto de características a utilizar es el de 1.1.5. El desarrollo a seguir es similar al expuesto en 1.1.4.
Luego se se evalúo el mejor clasificador obtenido, para lo cual se hizo uso de dos métricas: la matriz de confusión y el accuracy. Ambas fueron calculadas mediante una predicción sobre el subconjunto de validación.
El código utilizado para el entrenamiento del clasificador SVM con kernel lineal con características de *SelectFromModel* se adjunta en 6.

Código 6: Obtención del mejor clasificador SVM lineal según las características obtenidas mediante *SelectFromModel*.

```
XTrainNormalizedSelected1 = Selector1.transform(XTrainNormalized)
XValidationNormalizedSelected1 = Selector1.transform(XValidationNormalized)
GridSearch1 = GridSearchCV(estimator=SVM, param_grid=ParametersGrid1, cv=
    ↪ PreDefinedSplit)
InitialTrainingTime = time.time()
```

```
GridSearch1.fit(np.vstack((XTrainNormalizedSelected1, XValidationNormalizedSelected1)),
    ↪ np.hstack((yTrain, yValidation)))
FinalTrainingTime = time.time()
BestSVM1 = GridSearch1.best_estimator_
TrainingTime1 = FinalTrainingTime - InitialTrainingTime
BestSVM1.fit(XTrainNormalizedSelected1, yTrain)
yPredictedValidation1 = BestSVM1.predict(XValidationNormalizedSelected1)
```

Los resultados obtenidos mediante la ejecución de 6 se presentan en 3.

Tabla 3: Mejor clasificador SVM lineal según *SelectFromModel* y su tiempo de entrenamiento.

Mejor clasificador SVM lineal	Tiempo de entrenamiento
SVC(C=0.1, kernel=linear)	0.04792594909667969

Las métricas obtenidas se adjuntan a continuación.

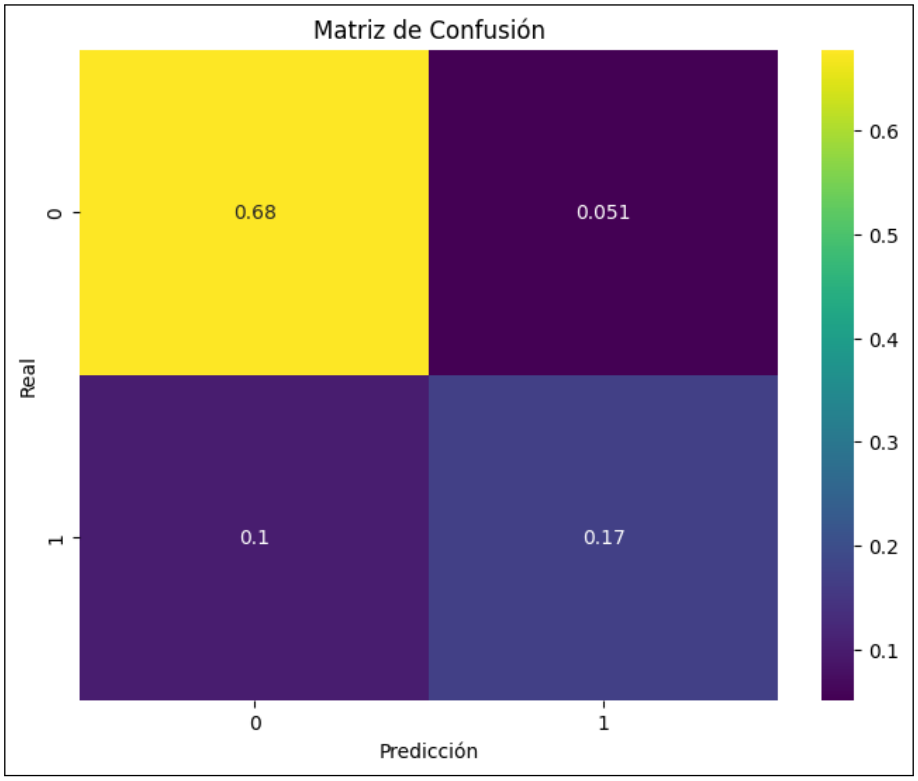


Figura 2: Matriz de confusión normalizada del mejor clasificador SVM lineal según *SelectFromModel*.

Tabla 4: Accuracy SVM con *SelectFromModel*.

Accuracy
0.847457627118644

De la matriz de confusión normalizada, representada en 2, es posible observar que el valor de TN es superior al resto, incluso representando más de la mitad del total de casos, le sigue el valor de TP, luego el valor de FN y por ultimo el de FP, por lo que el clasificador obtenido es «bueno», ya que los primeros dos valores mencionados son aciertos y suman 85 %, mientras que el 15 % restante son errores.
Del valor del accuracy, representado en 4, es posible observar que es bastante alto, lo cual implica un clasificador «bueno».

1.1.8. h)

En la presente sub-sección se repetirá lo realizado en 1.1.7, pero ahora utilizando como selector de características *SelectKBest* de *scikit-learn*. Se utilizara $k = 6$ como parámetro de *SelectKBest*.
El código utilizado para el entrenamiento del clasificador SVM con kernel lineal con características de *SelectKbest* con $k = 6$se adjunta en 7.

Código 7: Obtención del mejor clasificador SVM lineal según las características obtenidas mediante *SelectKBest* con $k = 6$.

```
Selector2 = SelectKBest(f_classif, k=6)
XTrainNormalizedSelected2 = Selector2.fit_transform(XTrainNormalized, yTrain)
XValidationNormalizedSelected2 = Selector2.fit_transform(XValidationNormalized,
    ↪ yValidation)
GridSearch2 = GridSearchCV(estimator=SVM, param_grid=ParametersGrid1, cv=
    ↪ PreDefinedSplit)
InitialTrainingTime = time.time()
GridSearch2.fit(np.vstack((XTrainNormalizedSelected2, XValidationNormalizedSelected2)),
    ↪ np.hstack((yTrain, yValidation)))
FinalTrainingTime = time.time()
BestSVM2 = GridSearch2.best_estimator_
TrainingTime2 = FinalTrainingTime - InitialTrainingTime
BestSVM2.fit(XTrainNormalizedSelected2, yTrain)
yPredictedValidation2 = BestSVM2.predict(XValidationNormalizedSelected2)
```

Los resultados obtenidos mediante la ejecución de 7 se presentan en 5.

Tabla 5: Mejor clasificador SVM lineal según las características de *SelectKBest* (con $k = 6$) y su tiempo de entrenamiento.

Mejor clasificador SVM lineal	Tiempo de entrenamiento
SVC(C=0.1, kernel=linear)	0.035726308822631836

Las características seleccionadas son:

1. sex.
2. thalach.
3. exang.
4. oldpeak.
5. ca.
6. thal.

Las metricas obtenidas se adjuntan a continuación.

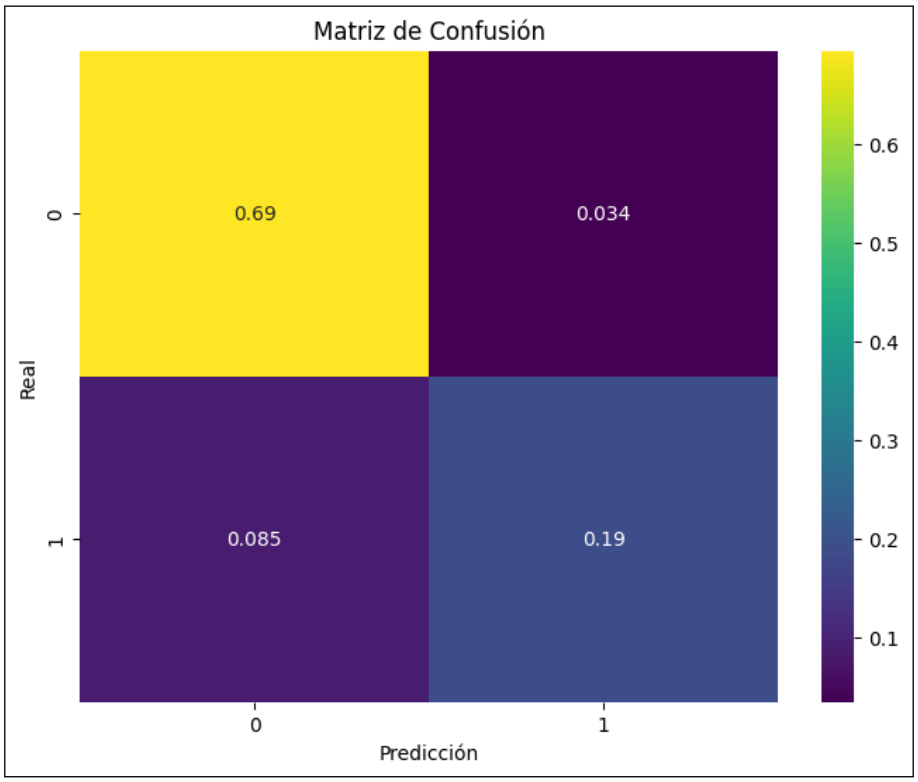


Figura 3: Matriz de confusión normalizada del mejor clasificador SVM lineal según las características de *SelectKBest* con $k = 6$.

Tabla 6: Accuracy SVM con *SelectKBest* con $k = 6$.

Accuracy
0.8813559322033898

De la matriz de confusión normalizada, representada en 3, es posible observar que el valor de TN es superior al resto, incluso representando más de la mitad del total de casos, le sigue el valor de TP, luego el valor de FN y por ultimo el de FP, por lo que el clasificador obtenido es «bueno», ya que los primeros dos valores mencionados son aciertos y suman 88 %, mientras que el 12 % restante son errores.
Del valor del accuracy, representado en 6, es posible observar que es bastante alto, lo cual implica un clasificador «bueno».

1.1.9. i)

En la presente sub-sección se repetira lo realizado en 1.1.7, pero ahora utilizando como selector de características *SelectKBest* de *scikit-learn*. Se utilizara $k = 3$ como parámetro de *SelectKBest*.
El código utilizado para el entrenamiento del clasificador SVM con kernel lineal con características de *SelectKbest* con $k = 3$ se adjunta en 8.

Código 8: Obtención del mejor clasificador SVM lineal según las características obtenidas mediante *SelectKBest* con $k = 3$.

```
Selector3 = SelectKBest(f_classif, k=3)
XTrainNormalizedSelected3 = Selector3.fit_transform(XTrainNormalized, yTrain)
XValidationNormalizedSelected3 = Selector3.fit_transform(XValidationNormalized,
    ↪ yValidation)
GridSearch3 = GridSearchCV(estimator=SVM, param_grid=ParametersGrid1, cv=
    ↪ PreDefinedSplit)
InitialTrainingTime= time.time()
GridSearch3.fit(np.vstack((XTrainNormalizedSelected3, XValidationNormalizedSelected3)),
    ↪ np.hstack((yTrain, yValidation)))
FinalTrainingTime = time.time()
BestSVM3 = GridSearch3.best_estimator_
TrainingTime3 = FinalTrainingTime - InitialTrainingTime
BestSVM3.fit(XTrainNormalizedSelected3, yTrain)
yPredictedValidation3 = BestSVM3.predict(XValidationNormalizedSelected3)
```

Los resultados obtenidos mediante la ejecución de 8 se presentan en 7.

Tabla 7: Mejor clasificador SVM lineal según las características de *SelectKBest* con $k = 3$ y su tiempo de entrenamiento.

Mejor clasificador SVM lineal	Tiempo de entrenamiento
SVC(C=0.1, kernel=linear)	0.02878403663635254

Las características seleccionadas son:

1. exang.
2. oldpeak.

3. ca.

Las métricas obtenidas se adjuntan a continuación.

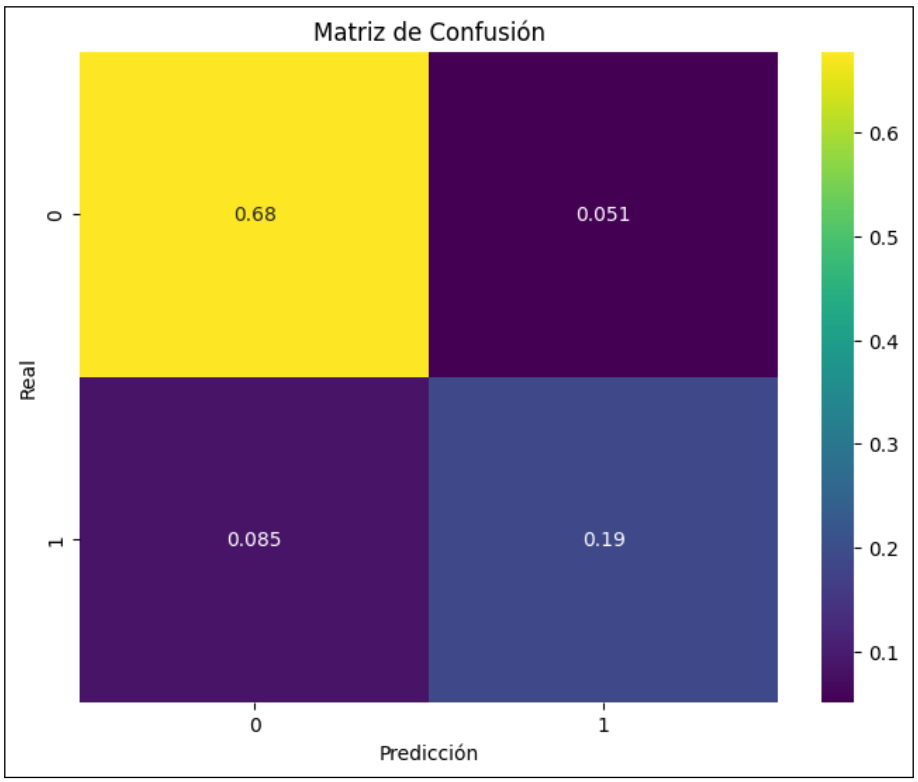


Figura 4: Matriz de confusión normalizada del mejor clasificador SVM lineal según las características de *SelectKBest* con $k = 3$.

Tabla 8: Accuracy con *SelectKBest* con $k = 3$.

Accuracy
0.864406779661017

De la matriz de confusión normalizada, representada en 4, es posible observar que el valor de TN es superior al resto, incluso representando más de la mitad del total de casos, le sigue el valor de TP, luego el valor de FN y por ultimo el de FP, por lo que el clasificador obtenido es «bueno», ya que los primeros dos valores mencionados son aciertos y suman 87 %, mientras que el 13 % restante son errores.
Del valor del accuracy, representado en 8, es posible observar que es bastante alto, lo cual implica un clasificador «bueno».

1.1.10. j)

En la presente sub-sección se repetirá el desarrollo de 1.1.4 y 1.1.5, pero utilizando un clasificador RandomForest, con una profundidad de 8 y utilizando una grilla sobre el hiperparámetro **n_estimators**.
El código utilizado para la creación y entrenamiento del clasificador RandomForest con max_depth = 8 se adjunta en 9.

Código 9: Obtención del mejor clasificador RandomForest con max_depth = 8.

```
RandomForest = RandomForestClassifier(max_depth=8, random_state=3)
ParametersGrid2 = {'n_estimators': [50, 100, 150, 200, 250]}
GridSearch4 = GridSearchCV(RandomForest, param_grid=ParametersGrid2, cv=
    ↪ PreDefinedSplit)
InitialTrainingTime = time.time()
GridSearch4.fit(np.vstack((XTrainNormalized, XValidationNormalized)), np.hstack((yTrain,
    ↪ yValidation)))
FinalTrainingTime = time.time()
BestRandomForest0 = GridSearch4.best_estimator_
BestScore = GridSearch4.best_score_
TrainingTime4 = FinalTrainingTime - InitialTrainingTime
BestRandomForest0.fit(XTrainNormalized, yTrain)
yPredictedValidation4 = BestRandomForest0.predict(XValidationNormalized)
```

Los resultados obtenidos mediante la ejecución de 9 se presentan en 9.

Tabla 9: Mejor clasificador RandomForest y su tiempo de entrenamiento.

Mejor clasificador RandomForest	Tiempo de entrenamiento
RandomForestClassifier (max_depth=8, n_estimators=100)	1.4202051162719727

Además, se evalúo el mejor clasificador obtenido en 9, para lo cual se hizo uso de dos métricas: la matriz de confusión y el accuracy. Ambas fueron calculadas mediante una predicción sobre el subconjunto de validación.
Lo obtenido se presenta a continuación.

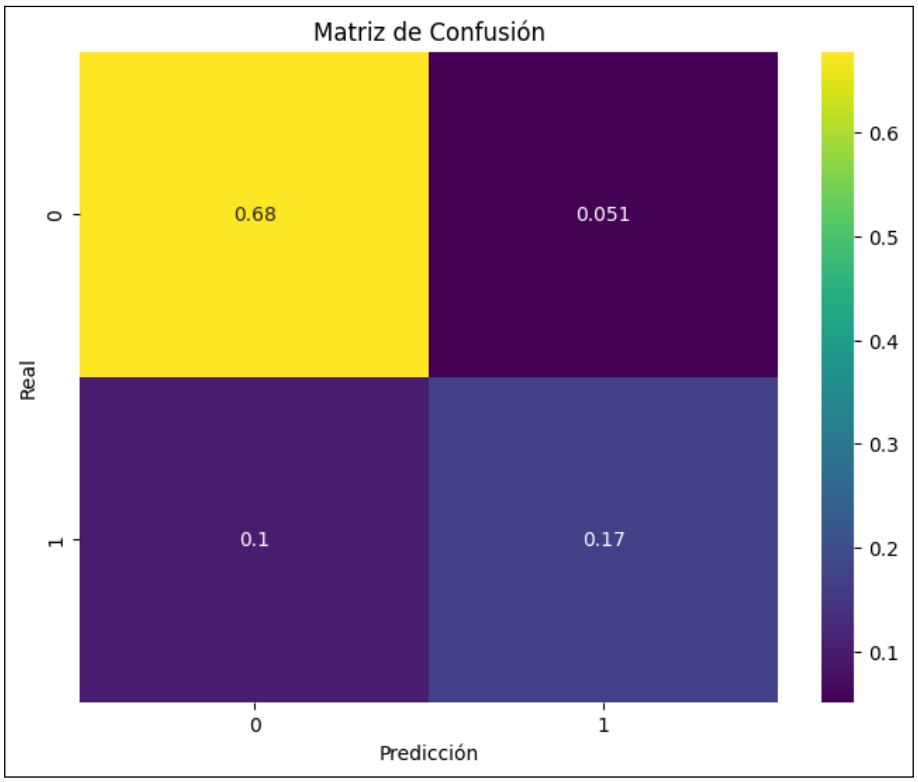


Figura 5: Matriz de confusión normalizada del mejor clasificador RandomForest.

Tabla 10: Accuracy RandomForest

Accuracy
0.847457627118644

De la matriz de confusión normalizada, representada en 5, es posible observar que el valor de TN es superior al resto, incluso representando más de la mitad del total de casos, le sigue el valor de TP, luego el valor de FN y por ultimo el de FP, por lo que el clasificador obtenido es «bueno», ya que los primeros dos valores mencionados son aciertos y suman 88 %, mientras que el 12 % restante son errores.
Del valor del accuracy, representado en 10, es posible observar que es bastante alto, lo cual implica un clasificador «bueno».

1.1.11. k)

En la presente sub-sección se repetira lo realizado en 1.1.7, pero ahora utilizando como clasificador al obtenido en 1.1.10.
El código utilizado para el entrenamiento del clasificador RandomForest con características de *SelectFromModel* se adjunta en 10.

Código 10: Obtención del mejor clasificador RandomForest según las características obtenidas mediante *SelectFromModel*.

```
Selector4 = SelectFromModel(BestRandomForest0)
XTrainNormalizedSelected4 = Selector4.transform(XTrainNormalized)
XValidationNormalizedSelected4 = Selector4.transform(XValidationNormalized)
GridSearch5 = GridSearchCV(RandomForest, param_grid=ParametersGrid2, cv=
    ↪ PreDefinedSplit)
InitialTrainingTime = time.time()
GridSearch5.fit(np.vstack((XTrainNormalizedSelected4, XValidationNormalizedSelected4)),
    ↪ np.hstack((yTrain, yValidation)))
FinalTrainingTime = time.time()
BestRandomForest1 = GridSearch5.best_estimator_
BestScore = GridSearch5.best_score_
TrainingTime5 = FinalTrainingTime - InitialTrainingTime
BestRandomForest1.fit(XTrainNormalizedSelected4, yTrain)
yPredictedValidation5 = BestRandomForest1.predict(XValidationNormalizedSelected4)
```

Los resultados obtenidos mediante la ejecución de 10 se presentan en 11.

Tabla 11: Mejor clasificador RandomForest con *SelectFromModel* y su tiempo de entrenamiento.

Mejor clasificador RandomForest	Tiempo de entrenamiento
RandomForestClassifier (max_depth=8, n_estimators=150)	1.4190337657928467

Las características seleccionadas son:

1. age.
2. trestbps.
3. chol.
4. thalach.
5. oldpeak.
6. ca.

Las métricas obtenidas se adjuntan a continuación.

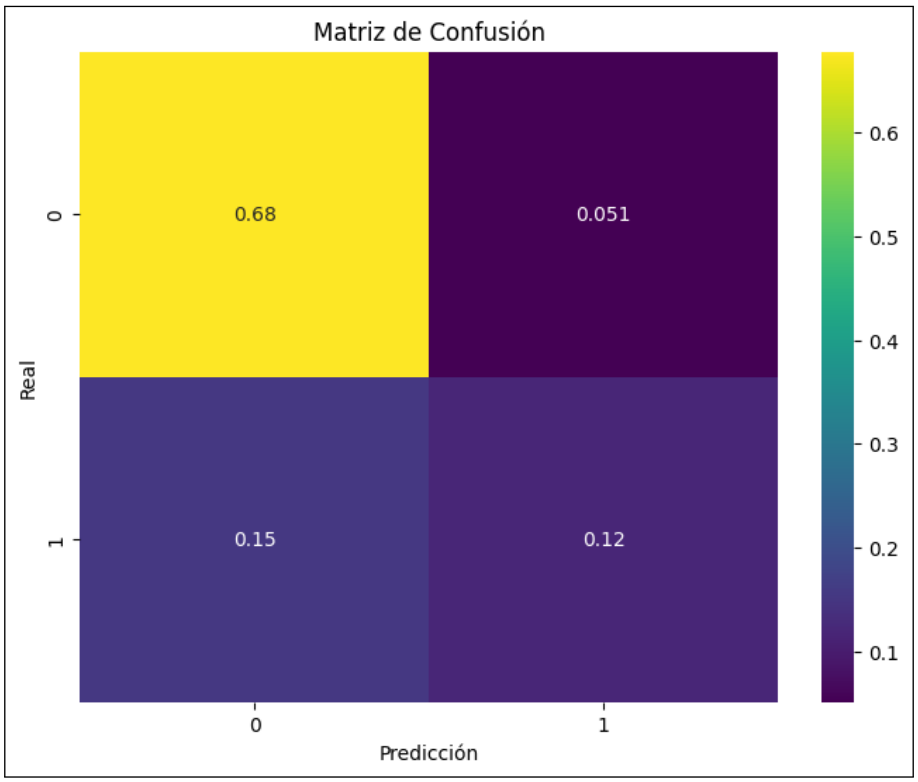


Figura 6: Matriz de confusión normalizada del mejor clasificador RandomForest con *SelectFromModel*.

Tabla 12: Accuracy RandomForest con *SelectFromModel*.

Accuracy
0.8813559322033898

De la matriz de confusión normalizada, representada en 6, es posible observar que el valor de TN es superior al resto, incluso representando más de la mitad del total de casos, le sigue el valor de TP, luego el valor de FN y por ultimo el de FP, por lo que el clasificador obtenido es «bueno», ya que los primeros dos valores mencionados son aciertos y suman 80 %, mientras que el 20 % restante son errores.
Del valor del accuracy, representado en 12, es posible observar que es alto, lo cual implica un clasificador «bueno».

1.1.12. l)

En la presente sub-sección se repetirá lo realizado en 1.1.8, pero ahora utilizando como clasificador un RandomForest.
El código utilizado para el entrenamiento del clasificador RandomForest con características de *SelectKbest* con $k = 6$ se adjunta en 11.

Código 11: Obtención del mejor clasificador RandomForest con *SelectKBest* con $k = 6$.

```
Selector5 = SelectKBest(f_classif, k=6)
XTrainNormalizedSelected5 = Selector5.fit_transform(XTrainNormalized, yTrain)
XValidationNormalizedSelected5 = Selector5.fit_transform(XValidationNormalized,
    ↪ yValidation)
GridSearch6 = GridSearchCV(RandomForest, param_grid=ParametersGrid2, cv=
    ↪ PreDefinedSplit)
InitialTrainingTime = time.time()
GridSearch6.fit(np.vstack((XTrainNormalizedSelected5, XValidationNormalizedSelected5)),
    ↪ np.hstack((yTrain, yValidation)))
FinalTrainingTime = time.time()
BestRandomForest2 = GridSearch6.best_estimator_
TrainingTime6 = FinalTrainingTime - InitialTrainingTime
BestRandomForest2.fit(XTrainNormalizedSelected5, yTrain)
yPredictedValidation6 = BestRandomForest2.predict(XValidationNormalizedSelected5)
```

Los resultados obtenidos mediante la ejecución de 11 se presentan en 13.

Tabla 13: Mejor clasificador RandomForest con *SelectKBest* con $k = 6$ y su tiempo de entrenamiento.

Mejor clasificador SVM lineal	Tiempo de entrenamiento
RandomForestClassifier (max_depth=8, n_estimators=50)	1.4290881156921387

Las características seleccionadas son:

1. sex.
2. thalach.
3. exang.
4. oldpeak.
5. ca.
6. thal.

Las métricas obtenidas se adjuntan a continuación.

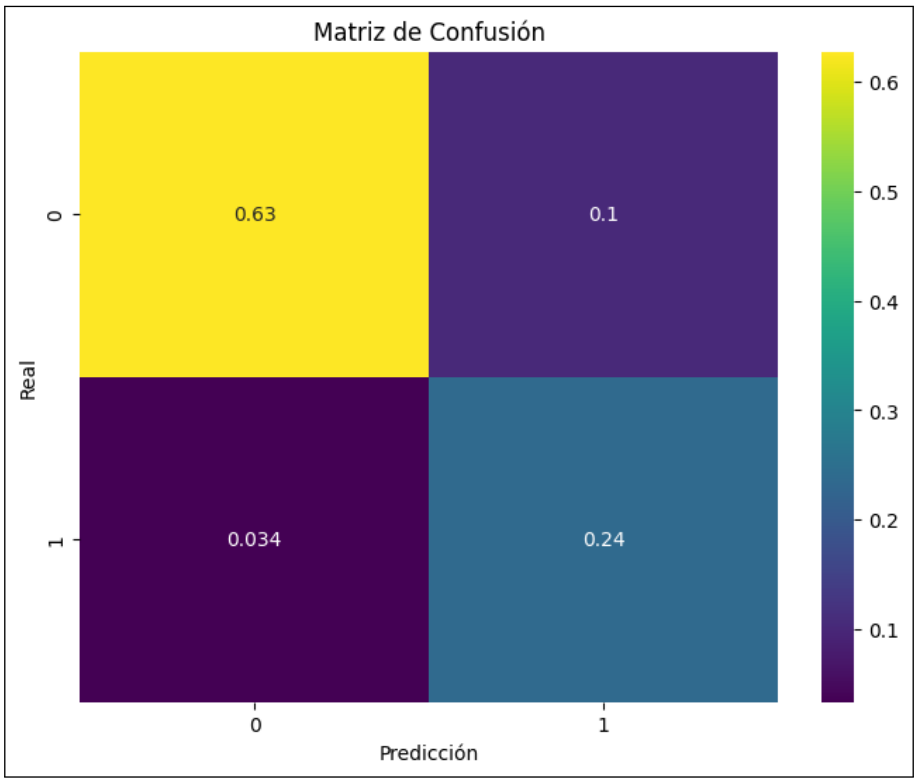


Figura 7: Matriz de confusión normalizada del mejor clasificador RandomForest con *SelectKBest* con $k = 6$.

Tabla 14: Accuracy RandomForest con *SelectKBest* con $k = 6$

Accuracy
0.864406779661017

De la matriz de confusión normalizada, representada en 7, es posible observar que el valor de TN es superior al resto, incluso representando más de la mitad del total de casos, le sigue el valor de TP, luego el valor de FP y por ultimo el de FN, por lo que el clasificador obtenido es «bueno», ya que los primeros dos valores mencionados son aciertos y suman 87 %, mientras que el 13 % restante son errores.
Del valor del accuracy, representado en 14, es posible observar que es bastante alto, lo cual implica un clasificador «bueno».

1.1.13. m)

En la presente sub-sección se repetirá lo realizado en 1.1.9, pero ahora utilizando como clasificador un RandomForest.
El código utilizado para el entrenamiento del clasificador RandomForest con características de *SelectKbest* (con $k = 3$) se adjunta en 12.

Código 12: Obtención del mejor clasificador RandomForest con *SelectKBest* con $k = 3$.

```
Selector6 = SelectKBest(f_classif, k=3)
XTrainNormalizedSelected6 = Selector6.fit_transform(XTrainNormalized, yTrain)
XValidationNormalizedSelected6 = Selector6.fit_transform(XValidationNormalized,
    ↪ yValidation)
GridSearch7 = GridSearchCV(RandomForest, param_grid=ParametersGrid2, cv=
    ↪ PreDefinedSplit)
InitialTrainingTime = time.time()
GridSearch7.fit(np.vstack((XTrainNormalizedSelected6, XValidationNormalizedSelected6)),
    ↪ np.hstack((yTrain, yValidation)))
FinalTrainingTime = time.time()
BestRandomForest3 = GridSearch7.best_estimator_
BestScore = GridSearch7.best_score_
TrainingTime7 = FinalTrainingTime - InitialTrainingTime
BestRandomForest3.fit(XTrainNormalizedSelected6, yTrain)
yPredictedValidation7 = BestRandomForest3.predict(XValidationNormalizedSelected6)
```

Los resultados obtenidos mediante la ejecución de 12 se presentan en 15.

Tabla 15: Mejor clasificador RandomForest con *SelectKBest* con $k = 3$ y su tiempo de entrenamiento.

Mejor clasificador SVM lineal	Tiempo de entrenamiento
RandomForestClassifier (max_depth=8, n_estimators=100)	2.105545997619629

Las características seleccionadas son:

1. exang.
2. oldpeak.
3. ca.

Las métricas obtenidas se adjuntan a continuación.

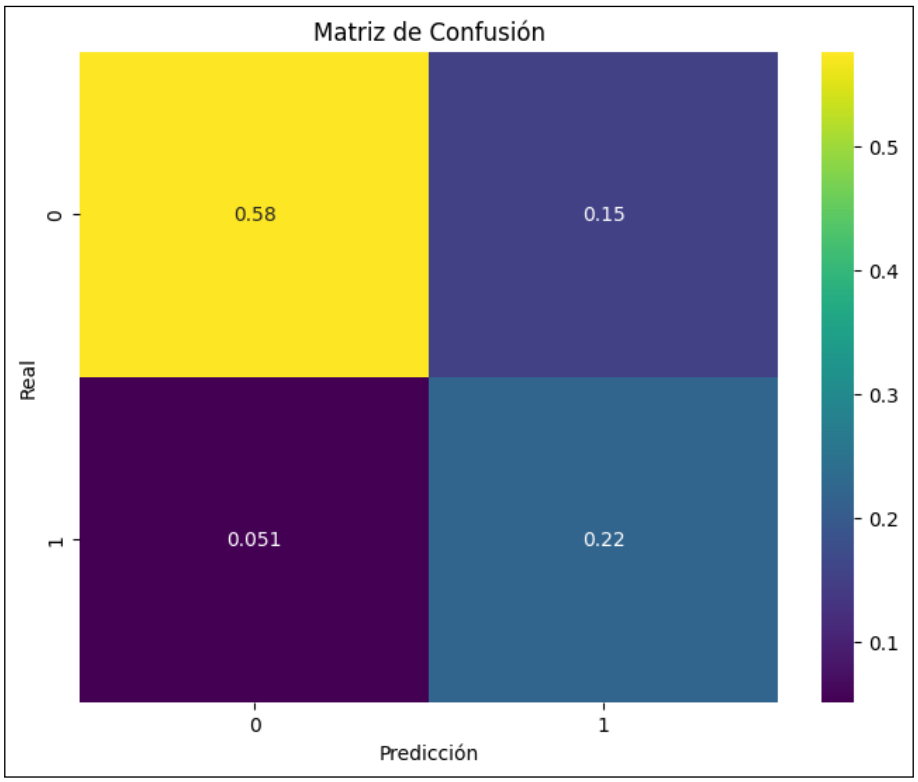


Figura 8: Matriz de confusión normalizada del mejor clasificador RandomForest con *SelectKBest* con $k = 3$.

Tabla 16: Accuracy RandomForest con *SelectKBest* con $k = 3$.

Accuracy
0.7966101694915254

De la matriz de confusión normalizada, representada en 8, es posible observar que el valor de TN es superior al resto, incluso representando más de la mitad del total de casos, le sigue el valor de TP, luego el valor de FP y por ultimo el de FN, por lo que el clasificador obtenido es «bueno», ya que los primeros dos valores mencionados son aciertos y suman 80 %, mientras que el 10 % restante son errores.
Del valor del accuracy, representado en 16, es posible observar que es alto, lo cual implica un clasificador «bueno».

1.1.14. n)

En la presente sub-sección se evaluaron los mejores clasificadores SVM y RandomForest encontrados al reducir las características. Ambos son evaluados sobre el conjunto de prueba, obteniendo sus matrices de confusión normalizadas y sus valores de accuracy.
Las métricas obtenidas se adjuntan a continuación.

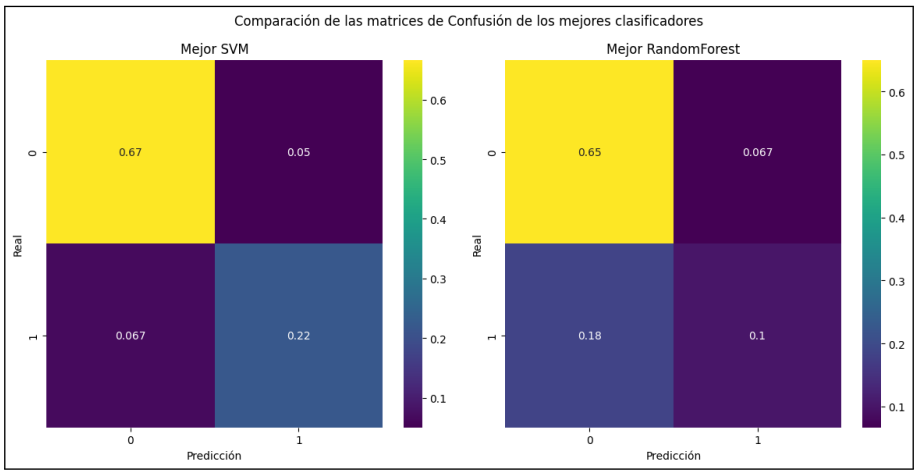


Figura 9: Comparación de las matrices de confusión de los mejores clasificadores.

Tabla 17: Comparación de Accuracy.

Accuracy SVM lineal	Accuracy RandomForest
0.8833333333333333	0.75

A partir de 9 y 17 es posible afirmar que el mejor clasificador es el SVM lineal, en particular el SVM lineal que utiliza las características obtenidas mediante *SelectFromModel*.

1.2. Análisis

1.2.1. a)

A partir del desarrollo previamente expuesto en 1.1, es claro que de manera general el clasificador SVM lineal entrega mejores resultados en el caso particular de este DataSet. Lo anterior es debido a que de manera general sus valores de accuracy son superiores a los del clasificador RandomForest.

1.2.2. b)

El efecto de reducir las características sobre el accuracy y el tiempo de entrenamiento se pueden vislumbrar en 10 y 11.

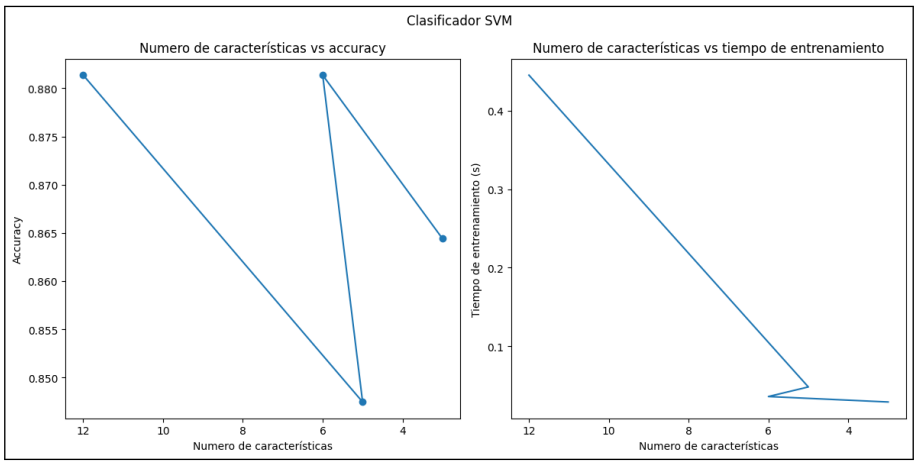


Figura 10: Gráficas para el clasificadores SVM.

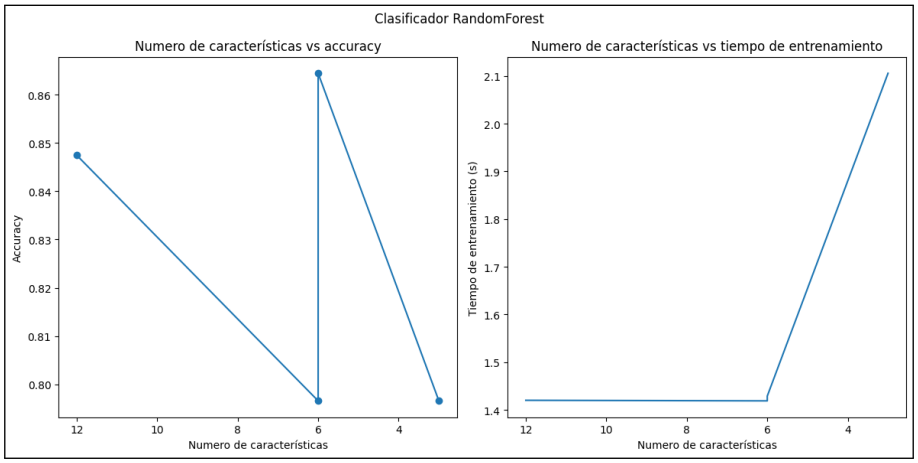


Figura 11: Gráficas para el clasificadores RandomForest.

En el caso del clasificador SVM, disminuir las características disminuye el accuracy, sobre todo en el caso en que se utiliza *SelectFromModel* y *SelectKBest* con $k = 3$. Al utilizar *SelectKBest* con $k = 6$ el accuracy obtenido es bastante similar al obtenido utilizando todas las características y podemos observar que el tiempo en dicho caso es bastante inferior, por lo que se podría considerar como una opción a utilizar si el tiempo es relevante. Acerca del tiempo de entrenamiento, al reducir las características este disminuye en todos los casos.
En el caso del clasificador RandomForest, disminuir las características aumenta el accuracy en el caso de utilizar *SelectKBest* con $k = 6$, pero disminuye el accuracy de manera significativa en los otros casos. Acerca del tiempo de entrenamiento, este es bastante plano para los 3 casos de estudio iniciales, pero para el caso de *SelectKBest* con $k = 3$ este aumenta.
Con todo lo anterior, es posible afirmar que para este Data Set y problema en particular, el mejor seleccionador de características es el *SelectKBest* con $k = 6$, pues este obtiene las mejores métricas en ambos clasificadores.

1.2.3. c)

A partir del desarrollo de la tarea y de los incisos anteriores, es posible afirmar que para el caso de los clasificador SVM no es útil disminuir las características, a menos que se quiera priorizar el tiempo de entrenamiento, pues su mayor valor de Accuracy es aquel que se obtiene utilizando las 12 características originales, mientras que para el caso del clasificador RandomForest, si es útil la reducción de características, pues su mayor valor de Accuracy es aquel que se obtiene utilizando *SelectKBest* con $k = 6$, y su tiempo de entrenamiento se ve levemente modificado.

2. Conclusiones

En la presente subsección se peresentaran las conclusiones generales del informe.

Aprendizajes

A continuación, se describen mis aprendizajes.

Accuracy He aprendido que el accuracy es una métrica útil para la comparación de clasificadores, cuando estos se utilizan sobre el mismo conjunto de datos. Pero, es importante que las clases estén balanceadas, pues de otra forma la métrica podría ser no tan fiable. Es por esto que es importante que se utilice con otras métricas.

Selección de características He aprendido que esta estrategia no siempre es útil, pues en algunos casos el rendimiento del clasificador se ve perjudicado al momento de implementarla. Lo anterior puede ocurrir si durante el proceso se eliminan características relevantes, el conjunto de datos contiene ruido o existen correlaciones entre características.
Además, he aprendido que existen tres métodos de selección de características, los cuales son:

1. Métodos de filtro.
2. Métodos de envoltura(Wrapper).
3. Métodos integrados.

En el caso particular de esta tarea, he utilizado los dos primeros.

RandomForest He aprendido acerca de la implementación de clasificadores a partir de los RandomForest, los cuales utilizan como base los arboles de decisión.

Dificultades

- Desconocimiento acerca del uso de los selectores de características, en particular, como acceder a las características seleccionadas por estos.

- Desconocimiento acerca de la implementación de un clasificador RandomForest.
- Desconocimiento del uso de random_state en *RandomForestClassifier*.

Evaluación de resultados

Los resultados obtenidos no son los que se esperan en función de la teoría, pues según https://archive.ics.uci.edu/dataset/45/heart+disease los valores de accuracy de los clasificadores RandomForest deberían, en su mayoría, ser superiores a los valores de accuracy de los clasificadores Support Vector Machines. Lo anterior es debido quizás a que hemos realizado una clasificación binaria, quizás si se realizara la clasificación con clases múltiples (la columna num original) los valores de accuracy serian los expuestos en la url.
Respecto a la selección de características los resultados son los que se esperan en función de la teoría, pues se observa como la utilidad de esta estrategia depende completamente de los clasificador utilizado y más aun del Data Set utilizado.

Código

Código A.1: Código de la tarea.

```
# -*- coding: utf-8 -*-
"""Tarea 3 – Selección de Características EL4106 - Diego Vega.ipynb

Automatically generated by Colab.

Original file is located at
   https://colab.research.google.com/drive/11nZC-zc5feO1KTsXADdzbYgSFBnjDsPN
"""

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import sklearn
import time
import seaborn as sns
from sklearn.model_selection import train_test_split, PredefinedSplit, GridSearchCV
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.feature_selection import SelectFromModel, SelectKBest, f_classif
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import confusion_matrix, roc_curve, auc, precision_recall_curve,
    ↪ average_precision_score, accuracy_score

"""# Selección de Características y Entrenamiento.

```

```
## a)
"""

DataSet = 'processed.cleveland.data'
Columns = ['age', 'sex', 'cp', 'trestbps', 'chol', 'fbs', 'restecg', 'thalach', 'exang', 'oldpeak', '
    ↪ slope', 'ca', 'thal', 'num']
DataFrame = pd.read_csv(DataSet, names=Columns)
def high_or_low(num):
  if num >= 2:
    return 1
  else:
    return 0
DataFrame['class'] = DataFrame['num'].apply(high_or_low)
DataFrame = DataFrame.drop('num', axis=1)
DataFrame = DataFrame.replace('?', np.nan)
DataFrame = DataFrame.dropna()

"""## b)"""

X = DataFrame.drop('class', axis=1)
y = DataFrame['class']
TrainRatio = 0.6
TestRatio = 0.2
ValidationRatio = 0.2
XTrain, XTemporary, yTrain, yTemporary = train_test_split(X, y, test_size=1-TrainRatio,
    ↪ random_state=3, stratify=y)
XValidation, XTest, yValidation, yTest = train_test_split(XTemporary, yTemporary,
    ↪ test_size=0.5, random_state=3, stratify=yTemporary)

"""## c)"""

Scaler = StandardScaler()
Scaler.fit(XTrain)
XTrainNormalized = Scaler.transform(XTrain)
XValidationNormalized = Scaler.transform(XValidation)
XTestNormalized = Scaler.transform(XTest)

"""## d)"""

Indexs = np.zeros(len(XTrainNormalized) + len(XValidationNormalized))
Indexs[:len(XTrainNormalized)] = -1
PreDefinedSplit = PredefinedSplit(test_fold=Indexs)
ParametersGrid1 = {'C': [0.01, 0.1, 1, 10, 100]}
SVM = SVC(kernel='linear')
GridSearch0 = GridSearchCV(estimator=SVM, param_grid=ParametersGrid1, cv=
    ↪ PreDefinedSplit)
InitialTrainingTime = time.time()
```

```
GridSearch0.fit(np.vstack((XTrainNormalized, XValidationNormalized)), np.hstack((yTrain,
    ↪ yValidation)))
FinalTrainingTime = time.time()
BestSVM0 = GridSearch0.best_estimator_
BestSVM0.fit(XTrainNormalized, yTrain)
TrainingTime0 = FinalTrainingTime - InitialTrainingTime
print(f'Tiempo de entrenamiento: {TrainingTime0}')
print(f'Mejor SVM: {BestSVM0}')

"""## e)"""

yPredictedValidation = BestSVM0.predict(XValidationNormalized)
ConfusionMatrix = confusion_matrix(yValidation, yPredictedValidation, normalize='all')
ValidationScore0 = accuracy_score(yPredictedValidation, yValidation)
print(f'Accuracy del mejor SVM sobre el conjunto de validación: {ValidationScore0}')
plt.figure(figsize=(8,6))
sns.heatmap(ConfusionMatrix, annot=True, cmap=plt.cm.viridis)
plt.title('Matriz de Confusión')
plt.xlabel('Predicción')
plt.ylabel('Real')
plt.show()

"""## f)"""

Selector1 = SelectFromModel(BestSVM0)
SelectedFeatures1 = X.columns[Selector1.get_support()]
print('Las features seleccionadas por SelectFromModel son:')
for i, Feature in enumerate(SelectedFeatures1):
  print(f'{i+1} {Feature}')

"""## g)"""

XTrainNormalizedSelected1 = Selector1.transform(XTrainNormalized)
XValidationNormalizedSelected1 = Selector1.transform(XValidationNormalized)
GridSearch1 = GridSearchCV(estimator=SVM, param_grid=ParametersGrid1, cv=
    ↪ PreDefinedSplit)
InitialTrainingTime = time.time()
GridSearch1.fit(np.vstack((XTrainNormalizedSelected1, XValidationNormalizedSelected1)),
    ↪ np.hstack((yTrain, yValidation)))
FinalTrainingTime = time.time()
BestSVM1 = GridSearch1.best_estimator_
TrainingTime1 = FinalTrainingTime - InitialTrainingTime
print(f'Tiempo de entrenamiento: {TrainingTime1}')
print(f'Mejor SVM: {BestSVM1}')
BestSVM1.fit(XTrainNormalizedSelected1, yTrain)
yPredictedValidation1 = BestSVM1.predict(XValidationNormalizedSelected1)
Labels = [0, 1]
```

```
ConfusionMatrix = confusion_matrix(yValidation, yPredictedValidation1, normalize='all')
ValidationScore1 = accuracy_score(yPredictedValidation1, yValidation)
print(f'Accuracy del mejor SVM sobre el conjunto de validación: {ValidationScore1}')
plt.figure(figsize=(8,6))
sns.heatmap(ConfusionMatrix, annot=True, cmap=plt.cm.viridis)
plt.title('Matriz de Confusión')
plt.xlabel('Predicción')
plt.ylabel('Real')
plt.show()

"""## h)"""

Selector2 = SelectKBest(f_classif, k=6)
XTrainNormalizedSelected2 = Selector2.fit_transform(XTrainNormalized, yTrain)
XValidationNormalizedSelected2 = Selector2.fit_transform(XValidationNormalized,
    ↪ yValidation)
GridSearch2 = GridSearchCV(estimator=SVM, param_grid=ParametersGrid1, cv=
    ↪ PreDefinedSplit)
InitialTrainingTime = time.time()
GridSearch2.fit(np.vstack((XTrainNormalizedSelected2, XValidationNormalizedSelected2)),
    ↪ np.hstack((yTrain, yValidation)))
FinalTrainingTime = time.time()
BestSVM2 = GridSearch2.best_estimator_
TrainingTime2 = FinalTrainingTime - InitialTrainingTime
print(f'Tiempo de entrenamiento: {TrainingTime2}')
print(f'Mejor SVM: {BestSVM2}')
BestSVM2.fit(XTrainNormalizedSelected2, yTrain)
yPredictedValidation2 = BestSVM2.predict(XValidationNormalizedSelected2)
ConfusionMatrix = confusion_matrix(yValidation, yPredictedValidation2, normalize='all')
ValidationScore2 = accuracy_score(yPredictedValidation2, yValidation)
print(f'Accuracy del mejor SVM sobre el conjunto de validación: {ValidationScore2}')
SelectedFeatures2 = X.columns[Selector2.get_support()]
print('Las features seleccionadas por SelectKBest (6) son:')
for i, Feature in enumerate(SelectedFeatures2):
  print(f'{i+1} {Feature}')
plt.figure(figsize=(8,6))
sns.heatmap(ConfusionMatrix, annot=True, cmap=plt.cm.viridis)
plt.title('Matriz de Confusión')
plt.xlabel('Predicción')
plt.ylabel('Real')
plt.show()

"""## i)"""

Selector3 = SelectKBest(f_classif, k=3)
XTrainNormalizedSelected3 = Selector3.fit_transform(XTrainNormalized, yTrain)
```

```
XValidationNormalizedSelected3 = Selector3.fit_transform(XValidationNormalized, yValidation)
GridSearch3 = GridSearchCV(estimator=SVM, param_grid=ParametersGrid1, cv=PreDefinedSplit)
InitialTrainingTime= time.time()
GridSearch3.fit(np.vstack((XTrainNormalizedSelected3, XValidationNormalizedSelected3)), np.hstack((yTrain, yValidation)))
FinalTrainingTime = time.time()
BestSVM3 = GridSearch3.best_estimator_
TrainingTime3 = FinalTrainingTime - InitialTrainingTime
print(f'Tiempo de entrenamiento: {TrainingTime3}')
print(f'Mejor SVM: {BestSVM3}')
BestSVM3.fit(XTrainNormalizedSelected3, yTrain)
yPredictedValidation3 = BestSVM3.predict(XValidationNormalizedSelected3)
Labels = [0, 1]
ConfusionMatrix = confusion_matrix(yValidation, yPredictedValidation3, normalize='all')
ValidationScore3 = accuracy_score(yPredictedValidation3, yValidation)
print(f'Accuracy del mejor SVM sobre el conjunto de validación: {ValidationScore3}')
SelectedFeatures3 = X.columns[Selector3.get_support()]
print('Las features seleccionadas por SelectKBest (3) son:')
for i, Feature in enumerate(SelectedFeatures3):
  print(f'{i+1} {Feature}')
plt.figure(figsize=(8,6))
sns.heatmap(ConfusionMatrix, annot=True, cmap=plt.cm.viridis)
plt.title('Matriz de Confusión')
plt.xlabel('Predicción')
plt.ylabel('Real')
plt.show()

"""## j)"""

RandomForest = RandomForestClassifier(max_depth=8, random_state=3)
ParametersGrid2 = {'n_estimators': [50, 100, 150, 200, 250]}
GridSearch4 = GridSearchCV(RandomForest, param_grid=ParametersGrid2, cv=PreDefinedSplit)
InitialTrainingTime = time.time()
GridSearch4.fit(np.vstack((XTrainNormalized, XValidationNormalized)), np.hstack((yTrain, yValidation)))
FinalTrainingTime = time.time()
BestRandomForest0 = GridSearch4.best_estimator_
TrainingTime4 = FinalTrainingTime - InitialTrainingTime
print(f'Tiempo de entrenamiento: {TrainingTime4}')
print(f'Mejor RandomForest: RandomForestClassifier (max_depth={BestRandomForest0.max_depth}, n_estimators={BestRandomForest0.n_estimators})')
BestRandomForest0.fit(XTrainNormalized, yTrain)
yPredictedValidation4 = BestRandomForest0.predict(XValidationNormalized)
Labels = [0, 1]
```

```
ConfusionMatrix = confusion_matrix(yValidation, yPredictedValidation4, normalize='all')
ValidationScore4 = accuracy_score(yPredictedValidation4, yValidation)
print(f'Accuracy del mejor Random Forest sobre el conjunto de validación: {ValidationScore4 }')
plt.figure(figsize=(8,6))
sns.heatmap(ConfusionMatrix, annot=True, cmap=plt.cm.viridis)
plt.title('Matriz de Confusión')
plt.xlabel('Predicción')
plt.ylabel('Real')
plt.show()

"""## k)"""

Selector4 = SelectFromModel(BestRandomForest0)
XTrainNormalizedSelected4 = Selector4.transform(XTrainNormalized)
XValidationNormalizedSelected4 = Selector4.transform(XValidationNormalized)
GridSearch5 = GridSearchCV(RandomForest, param_grid=ParametersGrid2, cv= PreDefinedSplit)
InitialTrainingTime = time.time()
GridSearch5.fit(np.vstack((XTrainNormalizedSelected4, XValidationNormalizedSelected4)), np.hstack((yTrain, yValidation)))
FinalTrainingTime = time.time()
BestRandomForest1 = GridSearch5.best_estimator_
TrainingTime5 = FinalTrainingTime - InitialTrainingTime
print(f'Tiempo de entrenamiento: {TrainingTime5}')
print(f'Mejor RandomForest: RandomForestClassifier (max_depth={BestRandomForest1.max_depth}, n_estimators={BestRandomForest1.n_estimators})')
BestRandomForest1.fit(XTrainNormalizedSelected4, yTrain)
yPredictedValidation5 = BestRandomForest1.predict(XValidationNormalizedSelected4)
Labels = [0, 1]
ConfusionMatrix = confusion_matrix(yValidation, yPredictedValidation5, normalize='all')
ValidationScore5 = accuracy_score(yPredictedValidation5, yValidation)
print(f'Accuracy del mejor RandomForest sobre el conjunto de validación: {ValidationScore5} ')
SelectedFeatures4 = X.columns[Selector4.get_support()]
print('Las features seleccionadas por SelectFromModel son:')
for i, Feature in enumerate(SelectedFeatures4):
  print(f'{i+1} {Feature}')
plt.figure(figsize=(8,6))
sns.heatmap(ConfusionMatrix, annot=True, cmap=plt.cm.viridis)
plt.title('Matriz de Confusión')
plt.xlabel('Predicción')
plt.ylabel('Real')
plt.show()

"""## l)"""

```

```
Selector5 = SelectKBest(f_classif, k=6)
XTrainNormalizedSelected5 = Selector5.fit_transform(XTrainNormalized, yTrain)
XValidationNormalizedSelected5 = Selector5.fit_transform(XValidationNormalized,
    ↪ yValidation)
GridSearch6 = GridSearchCV(RandomForest, param_grid=ParametersGrid2, cv=
    ↪ PreDefinedSplit)
InitialTrainingTime = time.time()
GridSearch6.fit(np.vstack((XTrainNormalizedSelected5, XValidationNormalizedSelected5)),
    ↪ np.hstack((yTrain, yValidation)))
FinalTrainingTime = time.time()
BestRandomForest2 = GridSearch6.best_estimator_
TrainingTime6 = FinalTrainingTime - InitialTrainingTime
print(f'Tiempo de entrenamiento: {TrainingTime6}')
print(f'Mejor RandomForest: RandomForestClassifier (max_depth={BestRandomForest2.
    ↪ max_depth}, n_estimators={BestRandomForest2.n_estimators})')
BestRandomForest2.fit(XTrainNormalizedSelected5, yTrain)
yPredictedValidation6 = BestRandomForest2.predict(XValidationNormalizedSelected5)
Labels = [0, 1]
ConfusionMatrix = confusion_matrix(yValidation, yPredictedValidation6, normalize='all')
ValidationScore6 = accuracy_score(yPredictedValidation6, yValidation)
print(f'Accuracy del mejor RandomForest sobre el conjunto de validación: {ValidationScore6}
    ↪ ')
SelectedFeatures5 = X.columns[Selector5.get_support()]
print('Las features seleccionadas por SelectKBest (6) son: ')
for i, Feature in enumerate(SelectedFeatures5):
  print(f'{i+1} {Feature}')
plt.figure(figsize=(8,6))
sns.heatmap(ConfusionMatrix, annot=True, cmap=plt.cm.viridis)
plt.title('Matriz de Confusión')
plt.xlabel('Predicción')
plt.ylabel('Real')
plt.show()

"""## m)"""

Selector6 = SelectKBest(f_classif, k=3)
XTrainNormalizedSelected6 = Selector6.fit_transform(XTrainNormalized, yTrain)
XValidationNormalizedSelected6 = Selector6.fit_transform(XValidationNormalized,
    ↪ yValidation)
GridSearch7 = GridSearchCV(RandomForest, param_grid=ParametersGrid2, cv=
    ↪ PreDefinedSplit)
InitialTrainingTime = time.time()
GridSearch7.fit(np.vstack((XTrainNormalizedSelected6, XValidationNormalizedSelected6)),
    ↪ np.hstack((yTrain, yValidation)))
FinalTrainingTime = time.time()
BestRandomForest3 = GridSearch7.best_estimator_
BestScore = GridSearch7.best_score_
```

```
277 TrainingTime7 = FinalTrainingTime - InitialTrainingTime
278 print(f'Tiempo de entrenamiento: {TrainingTime7}')
279 print(f'Mejor RandomForest: RandomForestClassifier (max_depth={BestRandomForest3.
        ↪ max_depth}, n_estimators={BestRandomForest3.n_estimators})')
280 BestRandomForest3.fit(XTrainNormalizedSelected6, yTrain)
281 yPredictedValidation7 = BestRandomForest3.predict(XValidationNormalizedSelected6)
282 ConfusionMatrix = confusion_matrix(yValidation, yPredictedValidation7, normalize='all')
283 ValidationScore7 = accuracy_score(yPredictedValidation7, yValidation)
284 print(f'Accuracy del mejor SVM sobre el conjunto de validación: {ValidationScore7}')
285 SelectedFeatures6 = X.columns[Selector6.get_support()]
286 for i, Feature in enumerate(SelectedFeatures6):
287   print(f'{i+1} {Feature}')
288 plt.figure(figsize=(8,6))
289 sns.heatmap(ConfusionMatrix, annot=True, cmap=plt.cm.viridis)
290 plt.title('Matriz de Confusión')
291 plt.xlabel('Predicción')
292 plt.ylabel('Real')
293 plt.show()
294
295 """## n)"""
296
297 BestSVM = BestSVM1
298 BestRandomForest = BestRandomForest2
299 XTestNormalizedScaled1 = Selector1.transform(XTestNormalized)
300 XTestNormalizedScaled2 = Selector4.transform(XTestNormalized)
301 Labels = [0, 1]
302 yPredictedTest1 = BestSVM.predict(XTestNormalizedScaled1)
303 ConfusionMatrix1 = confusion_matrix(yTest, yPredictedTest1, normalize='all')
304 TestScore1 = accuracy_score(yPredictedTest1, yTest)
305 print(f'Accuracy del mejor SVM sobre el conjunto de prueba: {TestScore1}')
306 yPredictedTest2 = BestRandomForest.predict(XTestNormalizedScaled2)
307 ConfusionMatrix2 = confusion_matrix(yTest, yPredictedTest2, normalize='all')
308 TestScore2 = accuracy_score(yPredictedTest2, yTest)
309 print(f'Accuracy del mejor RandomForest sobre el conjunto de prueba: {TestScore2}')
310 fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 6))
311 fig.suptitle('Comparación de las matrices de Confusión de los mejores clasificadores')
312 ax1 = sns.heatmap(ConfusionMatrix1, annot=True, cmap=plt.cm.viridis, ax=ax1)
313 ax1.set_title('Mejor SVM')
314 ax1.set_xlabel('Predicción')
315 ax1.set_ylabel('Real')
316 ax2 = sns.heatmap(ConfusionMatrix2, annot=True, cmap=plt.cm.viridis, ax=ax2)
317 ax2.set_title('Mejor RandomForest')
318 ax2.set_xlabel('Predicción')
319 ax2.set_ylabel('Real')
320 plt.tight_layout()
321 plt.show()
322
```

```
"""# Análisis

## a)

## b)
"""

NumberOfFeaturesSVM = [12, 5, 6, 3]
NumberOfFeaturesRandomForest = [12, 6, 6, 3]
TrainingTimesSVM = [TrainingTime0, TrainingTime1, TrainingTime2, TrainingTime3]
TrainingTimesRandomForest = [TrainingTime4, TrainingTime5, TrainingTime6,
    ↪ TrainingTime7]
AccuracysSVM = [ValidationScore0, ValidationScore1, ValidationScore2, ValidationScore3]
AccuracysRandomForest = [ValidationScore4, ValidationScore5, ValidationScore6,
    ↪ ValidationScore7]
plt.subplots(1, 2, figsize=(12, 6))
plt.suptitle('Clasificador SVM')
plt.subplot(1, 2, 1)
plt.title('Numero de características vs accuracy')
plt.plot(NumberOfFeaturesSVM, AccuracysSVM, marker='o')
plt.xlabel('Numero de características')
plt.ylabel('Accuracy')
plt.gca().invert_xaxis()
plt.subplot(1, 2, 2)
plt.title('Numero de características vs tiempo de entrenamiento')
plt.plot(NumberOfFeaturesSVM, TrainingTimesSVM)
plt.xlabel('Numero de características')
plt.ylabel('Tiempo de entrenamiento (s)')
plt.gca().invert_xaxis()
plt.tight_layout()
plt.show()
plt.subplots(1, 2, figsize=(12, 6))
plt.suptitle('Clasificador RandomForest')
plt.subplot(1, 2, 1)
plt.title('Numero de características vs accuracy')
plt.plot(NumberOfFeaturesRandomForest, AccuracysRandomForest, marker='o')
plt.xlabel('Numero de características')
plt.ylabel('Accuracy')
plt.gca().invert_xaxis()
plt.subplot(1, 2, 2)
plt.title('Numero de características vs tiempo de entrenamiento')
plt.plot(NumberOfFeaturesRandomForest, TrainingTimesRandomForest)
plt.xlabel('Numero de características')
plt.ylabel('Tiempo de entrenamiento (s)')
plt.gca().invert_xaxis()
plt.tight_layout()
plt.show()
```

```
368
369 """## c)"""
370
371 AccuracysSVMMaxIndex = AccuracysSVM.index(max(AccuracysSVM))
372 TrainingTimesSVMMinIndex = TrainingTimesSVM.index(min(TrainingTimesSVM))
373 NumberOfFeaturesSVMMaxAccuracy = NumberOfFeaturesSVM[AccuracysSVMMaxIndex]
374 NumberOfFeaturesSVMMinTime = NumberOfFeaturesSVM[TrainingTimesSVMMinIndex]
375 AccuracysRandomForestMaxIndex = AccuracysRandomForest.index(max(
    ↪ AccuracysRandomForest))
376 TrainingTimesRandomForestMinIndex = TrainingTimesRandomForest.index(min(
    ↪ TrainingTimesRandomForest))
377 NumberOfFeaturesRandomForestMaxAccuracy = NumberOfFeaturesSVM[
    ↪ AccuracysRandomForestMaxIndex]
378 NumberOfFeaturesRandomForestMinTime = NumberOfFeaturesSVM[
    ↪ TrainingTimesRandomForestMinIndex]
379 print(f'Para los clasificadores SVM, el mayor valor de Accuracy se obtiene con {
    ↪ NumberOfFeaturesSVMMaxAccuracy} caraterísticas y el menor tiempo de
    ↪ entrenamiento se obtiene con {NumberOfFeaturesSVMMinTime} caraterísticas.')
380 print(f'Para los clasificadores RandomForest, el mayor valor de Accuracy se obtiene con {
    ↪ NumberOfFeaturesRandomForestMaxAccuracy} caraterísticas y el menor tiempo de
    ↪ entrenamiento se obtiene con {NumberOfFeaturesRandomForestMinTime} caraterí
    ↪ sticas.')
381 print('Como el mayor valor de Accuracy se obtiene con las características originales, se puede
    ↪  afirmar que para dicho caso no es beneficioso la selección de características.')
382 print('Como el mayor valor de Accuracy y el menor tiempo de entrenamiento, para los
    ↪ clasificadores Random Forest, se obtiene con 6 características, se puede afirmar que
    ↪ para dicho caso es beneficioso la selección de características.')
```